match (say, *Java* and *Guide*). This is where the `minimum_should_match` attribute comes in handy.

The `minimum_should_match` attribute indicates the minimum number of words that should be used to match the documents. The next listing shows this in action.

Listing 10.8 Matching at least two words

```
GET books/_search
{
  "query": {
    "match": {
      "title": {
        "query": "Java Complete Guide",
        "operator": "OR",
        "minimum_should_match": 2
      }
    }
  }
}
```

Sets the minimum number of words that should match

This query matches at least two words (the `minimum_should_match` attribute is set to `2`) and fetches documents with a combination of two of the given three words. The `OR` operator is redundant here because it is applied by default.

NOTE Setting the value to `3` in the listing 10.8 is as good as changing the operator to `AND`: all the words must be matched.

10.5.6 Fixing typos using the fuzziness keyword

When searching for things, we sometimes incorrectly type the search criteria (we all have been there); for example, instead of searching for Java books, we may post a query with "Kava" as the search criterion. We know our intention is to search for Java books—and so does Elasticsearch.

Fuzziness makes character changes to the input string so it is the same as the string that may exist in the index. It employs the Levenshtein distance algorithm to fix incorrect spellings. We look at fuzziness further in section 10.12, but let's see how to use it with `match` queries.

A `match` query lets us add a `fuzziness` parameter to fix spelling mistakes. We can set it as a numeric value, where the expected value is `0`, `1`, or `2`, meaning zero, one, or two character changes (insertions, deletions, modifications), respectively. In addition to setting these values, we can use the `AUTO` setting to let the engine deal with the changes. The following listing shows how we can use `fuzziness` (with a value of `1`) to sort out our "Kava" typo.

Listing 10.9 Fixing a spelling mistake with the `fuzziness` parameter

```
GET books/_search
{
  "query": {
```